

Embedded Linux System

จัดทำโดย

นายเกียร มฤกูด
นายสถาพร สงวนวงษ์

อาจารย์ที่ปรึกษา

รศ. ดร. บุญธีร์ เครือตราฐ
ผศ. กฤตวัน ศิริบูรณ์

ความสำคัญและที่มา

Embedded System หรือ Real-time system มีจุดเริ่มต้นมาจากความต้องการเครื่องมือที่สามารถดูแลตัวเอง สามารถจัดการปัญหาต่างๆด้วยตัวเอง และสามารถตอบสนองต่อสิ่งที่เกิดขึ้นได้ในทันที โดยมีมนุษย์เป็นเพียงผู้สั่งให้เริ่มทำงานเท่านั้น

วัตถุประสงค์

- เพื่อศึกษาและทำความเข้าใจระบบสมองกลฝังตัว
(Embedded systems)
- เพื่อทราบถึงประสิทธิภาพในการทำงานของ Embedded Systems
ที่ใช้ Linux เป็นระบบปฏิบัติการ
- เพื่อศึกษาและทำความเข้าใจการทำงานของ Linux Device Driver
- เพื่อทำความเข้าใจการออกแบบการทำงานที่สอดคล้องกันระหว่าง
Hardware และ Software

ประโยชน์ที่คาดว่าจะได้รับ

- สามารถนำ Embedded Linux System ไปประยุกต์ใช้งานกับอุปกรณ์ประมวลผลแบบอื่นได้
- สามารถสร้าง Linux Device Driver ได้
- สามารถออกแบบ Linux Device Driver ให้เหมาะสมกับฮาร์ดแวร์เฉพาะอย่างได้
- สามารถนำหลักการทำงานไปประยุกต์ใช้กับระบบงานที่มีความซับซ้อนมากขึ้นได้

ทฤษฎีและแนวความคิดที่ใช้ในการทำงาน

ในการทดลองผู้จัดทำได้เลือกใช้ Debian Linux ซึ่งเป็นระบบปฏิบัติการประเภท Unix Clone Operating System ด้วยเหตุที่รองรับการทำงานกับอุปกรณ์ภายนอกได้มากมายหลายชนิด และจัดการได้ง่าย

ขอบเขตโครงการ

- ระบบจะต้องทำงานแบบฝังตัว
- โปรแกรมหลักถูกสร้างโดยมีลักษณะเป็น Linux module
- ใช้ระบบในการควบคุมเครน
- เครนจะต้องสามารถสั่งงานได้จากเครื่องคอมพิวเตอร์ระยะไกล
- ส่วนติดต่อกับผู้ใช้สามารถใช้ได้ทั้ง Web และ Command shell

ขอบเขตโครงการ (ต่อ)

- ตัวเครนสามารถเคลื่อนที่ได้ทั้งในแนวตั้งและแนวนอน
- เคนสามารถเคลื่อนที่โดยใช้ stepping motor
- ระบบสามารถแก้ไขปัญหาที่อาจเกิดขึ้นด้วยตัวเองได้ในระดับหนึ่ง
- ระบบมีส่วนแสดงสถานะในระหว่างการทำงาน

ทฤษฎีที่เกี่ยวข้อง

- ระบบที่ทำงานด้วยไมโครคอนโทรลเลอร์

เป็นระบบขนาดเล็ก ส่วนใหญ่จะมี *Memory* ในตัวและผู้ใช้ไม่สามารถโปรแกรมได้ เช่น *IC USB Mouse Controller*

- ระบบที่ทำงานด้วยไมโครโพรเซสเซอร์

เป็นระบบที่ใช้งานร่วมกับอุปกรณ์ภายนอก เช่น *Memory* โดยผู้ใช้สามารถทำการโปรแกรมได้ เช่น *Z80* เป็นต้น

ทฤษฎีที่เกี่ยวข้อง (ต่อ)

- ระบบขนาดใหญ่ (Large Scale)

เป็นระบบที่ประกอบด้วย *Microprocessor* มากกว่า 1 ตัว ส่วนใหญ่
จะทำงานด้วยระบบปฏิบัติการ

- *Unix Operating System*
- *Unix Clone Operating System*
- *Windows Operating System*

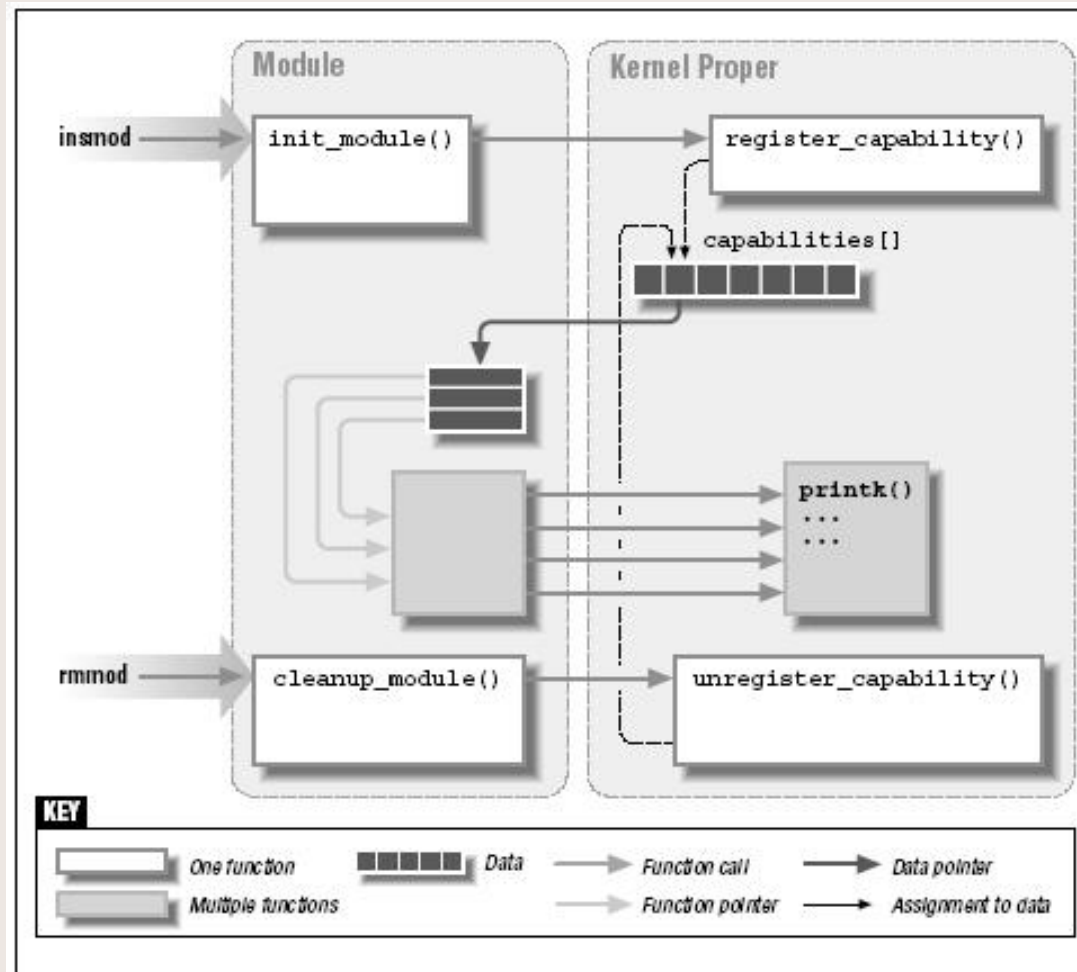
โครงสร้างของ Linux Kernel

Linux kernel เป็น โปรแกรมหลักของระบบที่ทำหน้าที่ดูแล
การใช้งาน โปรแกรมและ hardware ต่างๆของผู้ใช้ให้เป็นระเบียบ
และถูกต้อง โครงสร้างภายในของ Linux kernel ประกอบด้วยส่วน
ต่างๆ ดังนี้

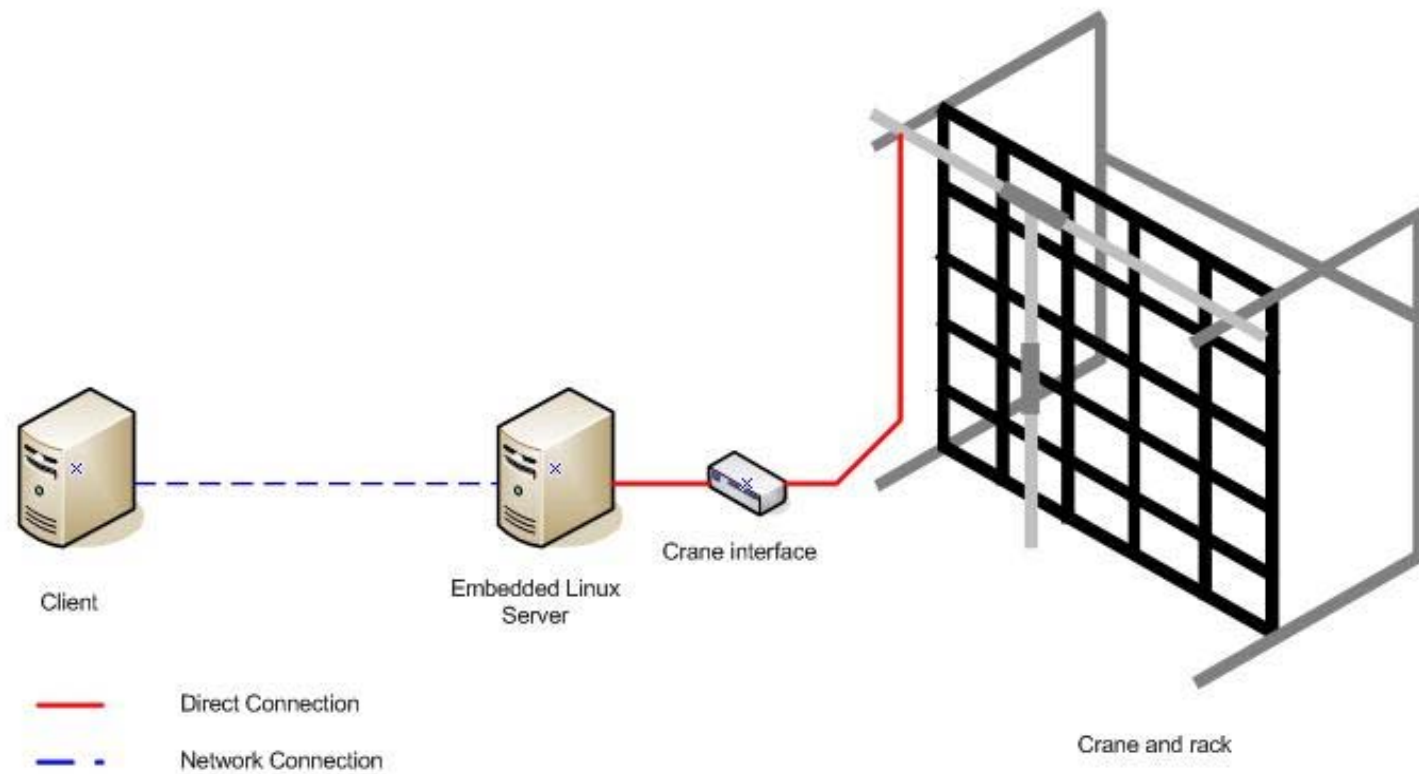
โครงสร้างของ Linux Kernel (ต่อ)

- Process management
- Memory management
- File system
- Device control
- Networking

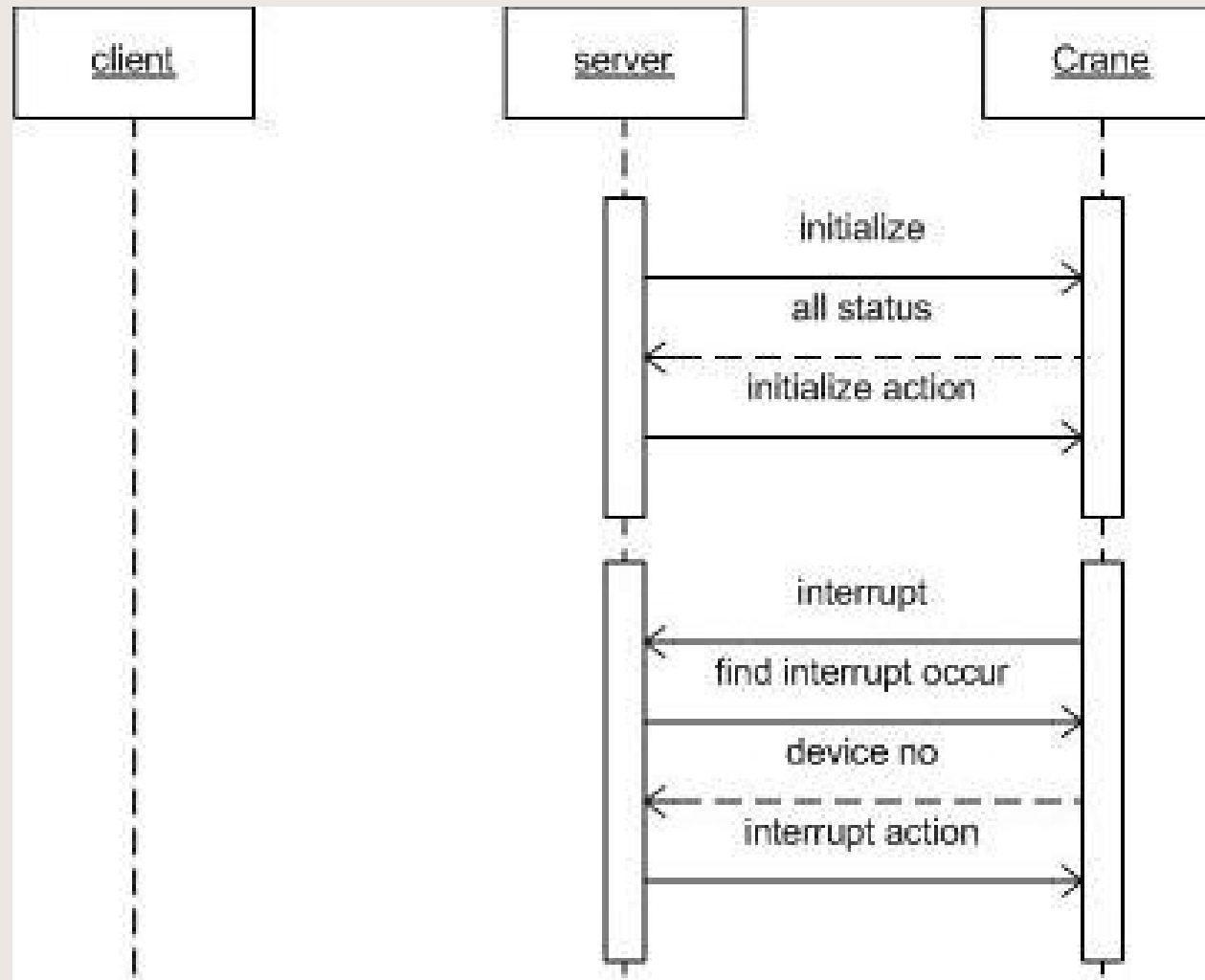
การทำงานกับ Module



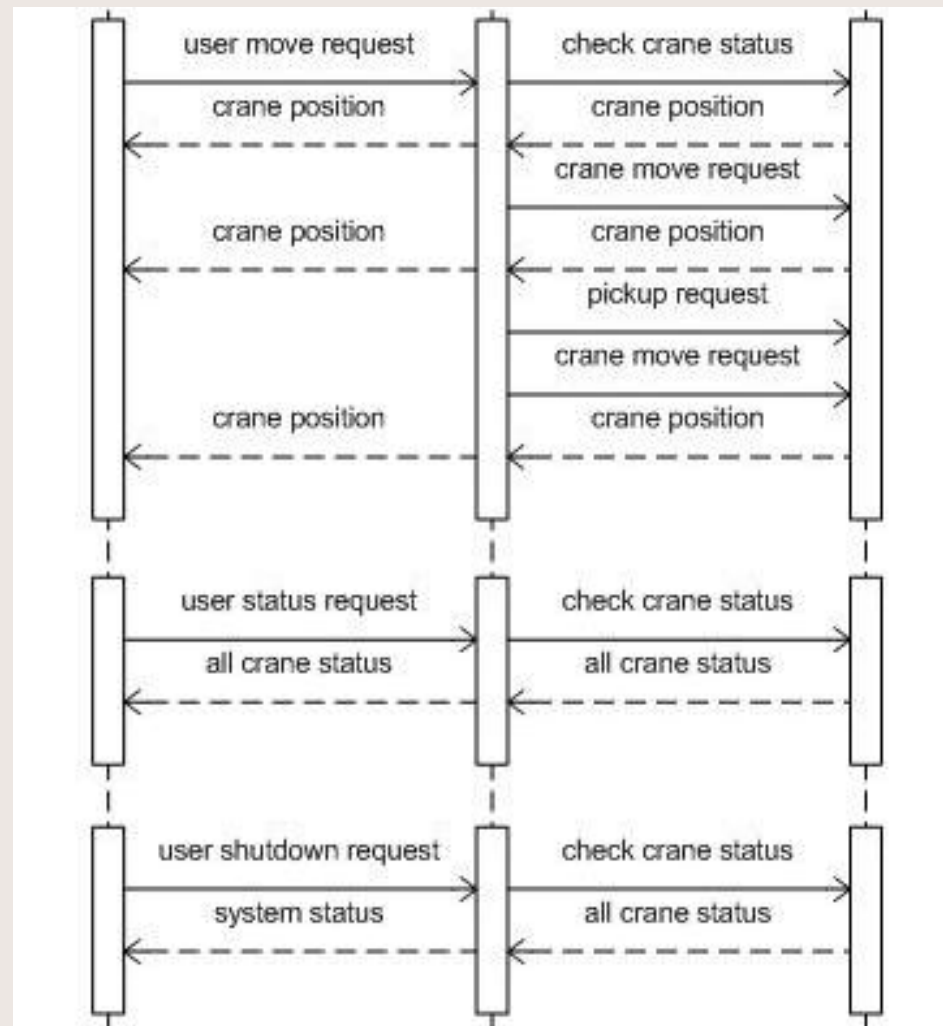
ภาพรวมของระบบ



Sequence Diagram



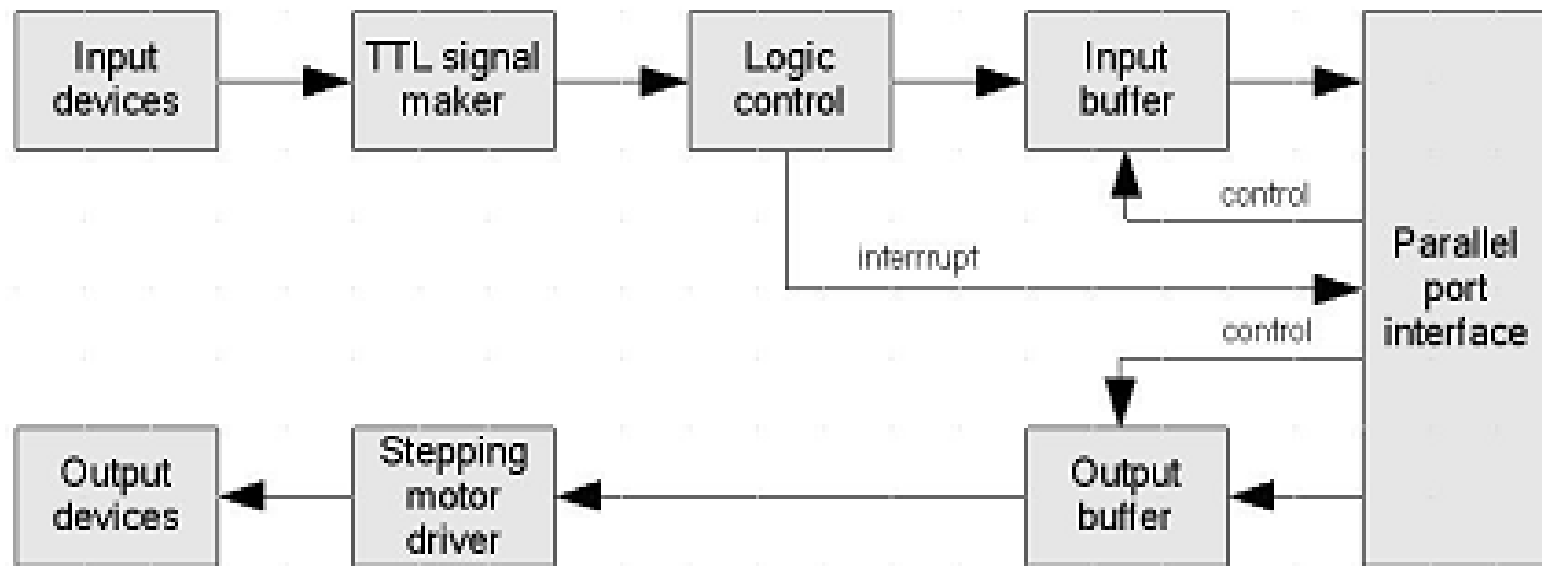
Sequence Diagram (ต่อ)



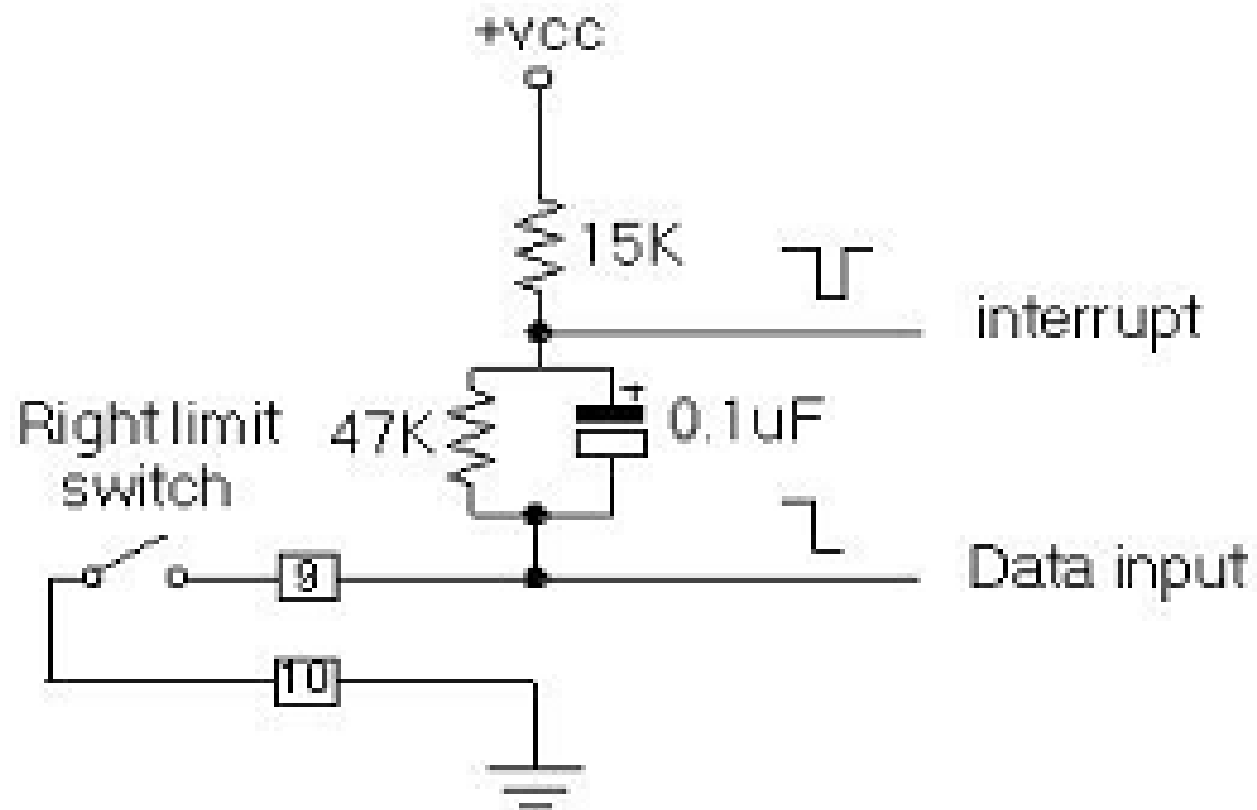
Server Hardware Specification

- CPU Pentium 200 MMX
- Memory 64 MB
- Harddisk ขนาด 8 GB
- Card LAN 10/100
- VGA Card PCI

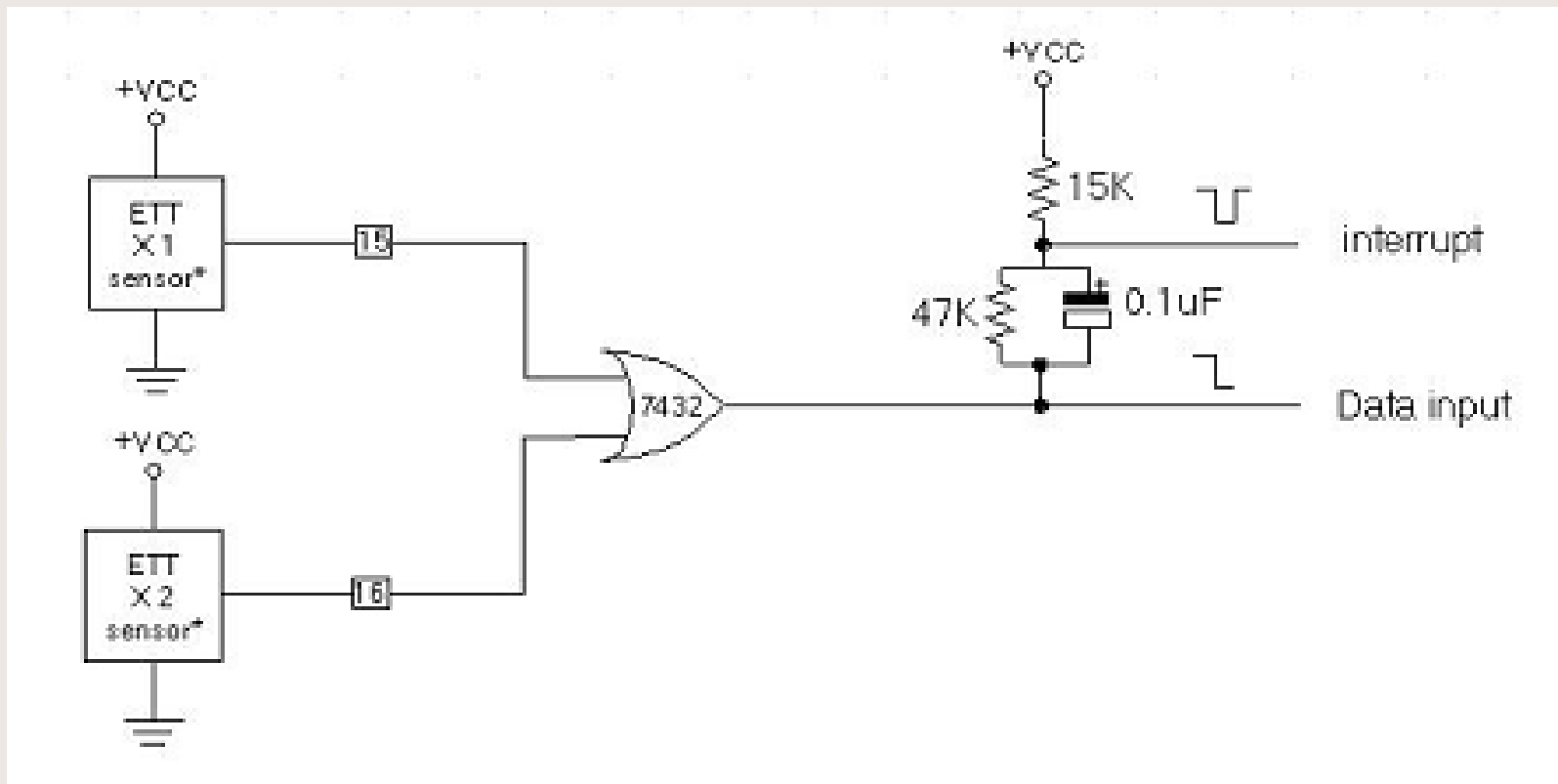
Block diagram of Crane interface



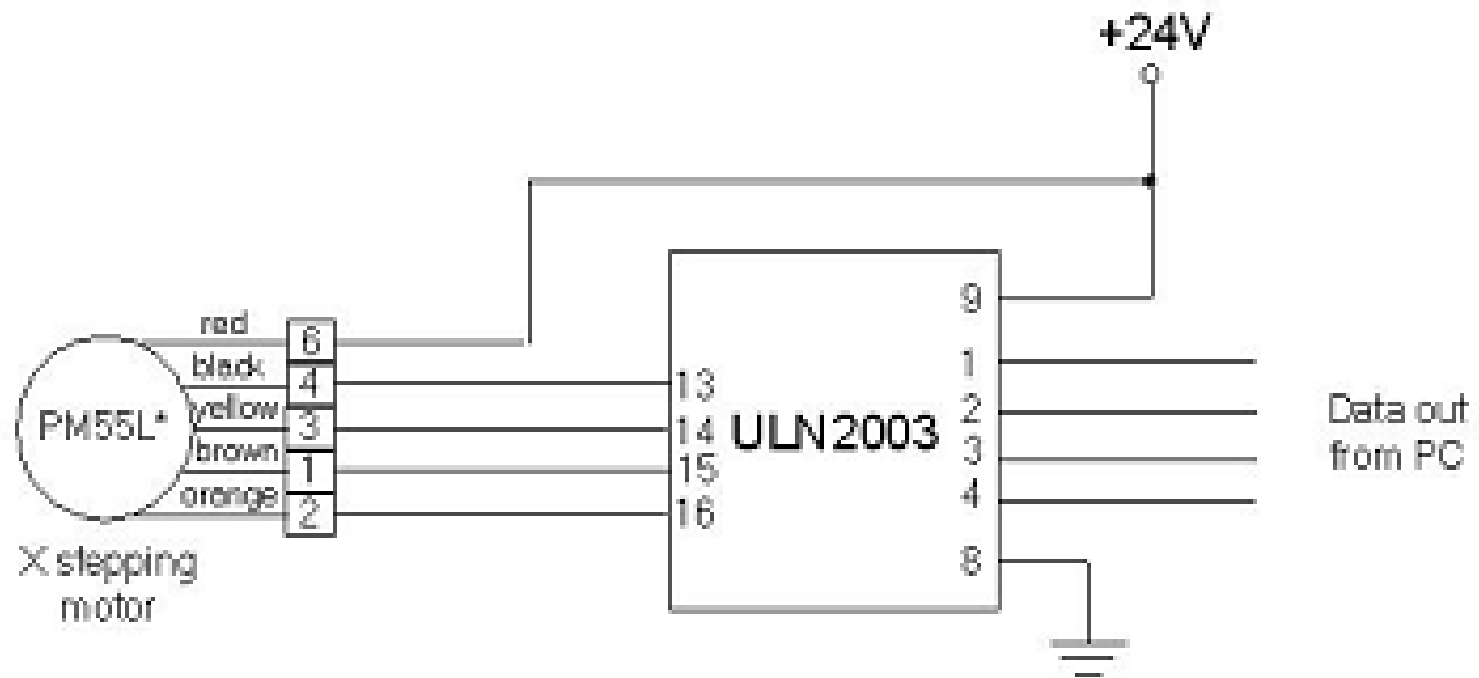
วงจร Dry Contact input



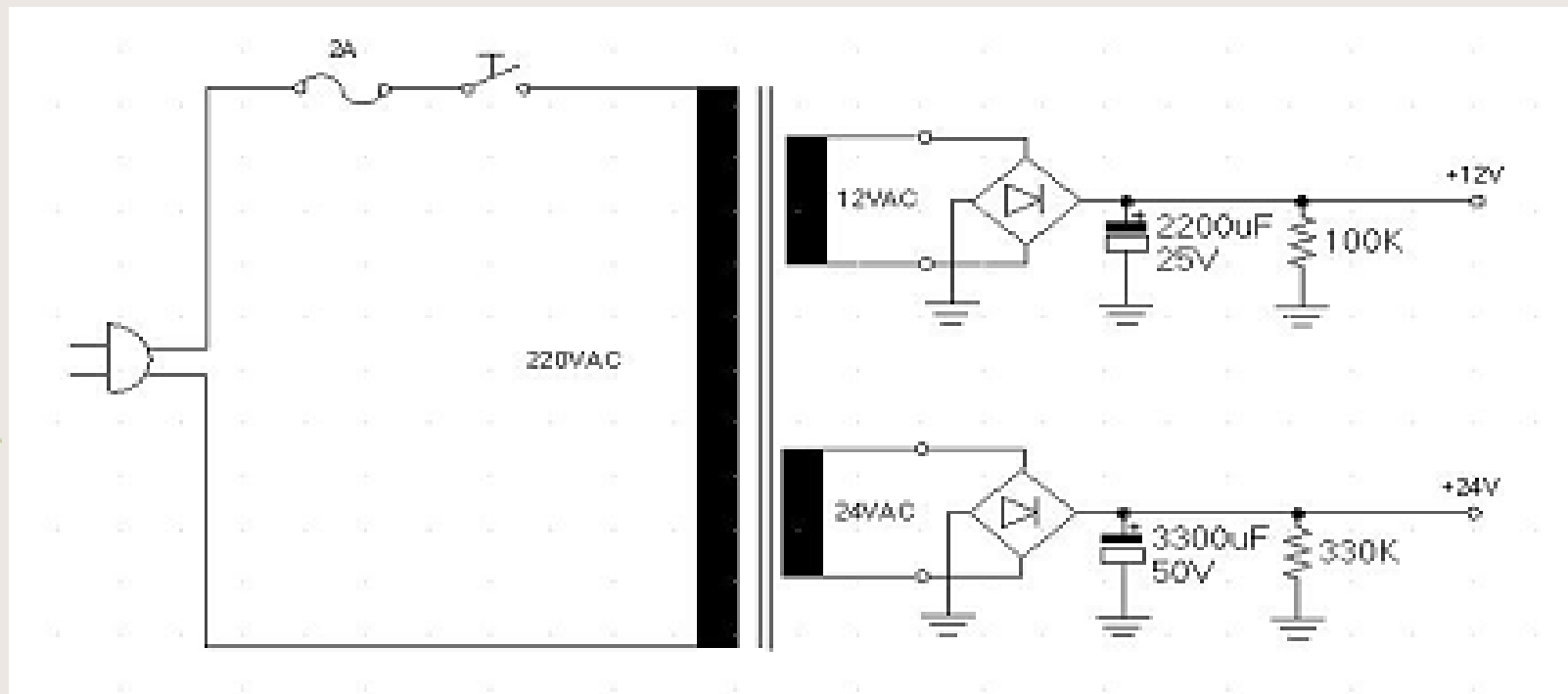
วงจรรับ Sensor input



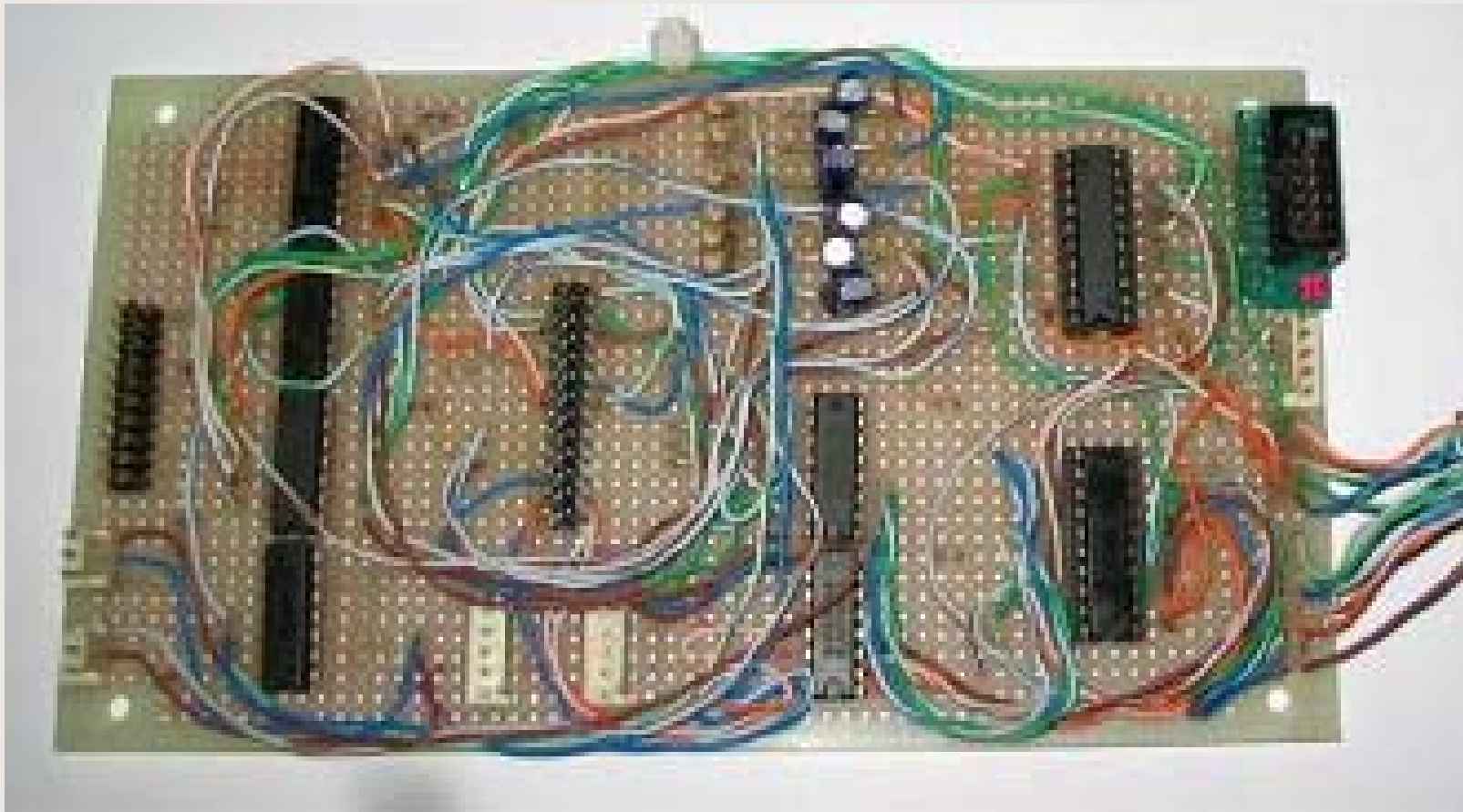
วงจรขับ Stepping Motor



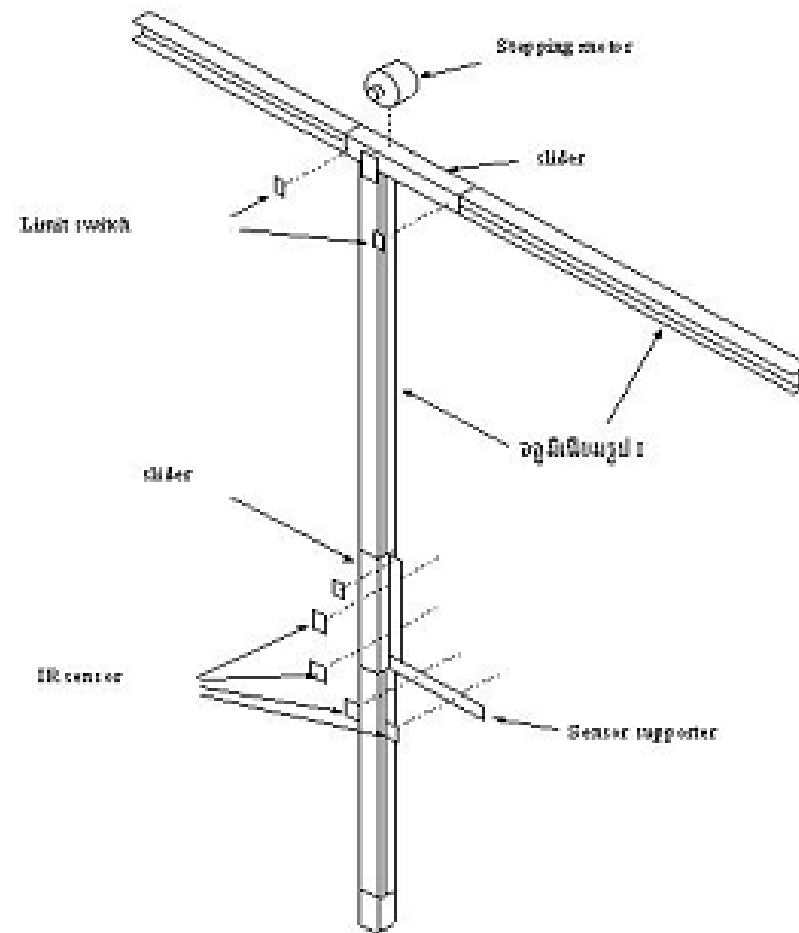
วงจร Power Supply



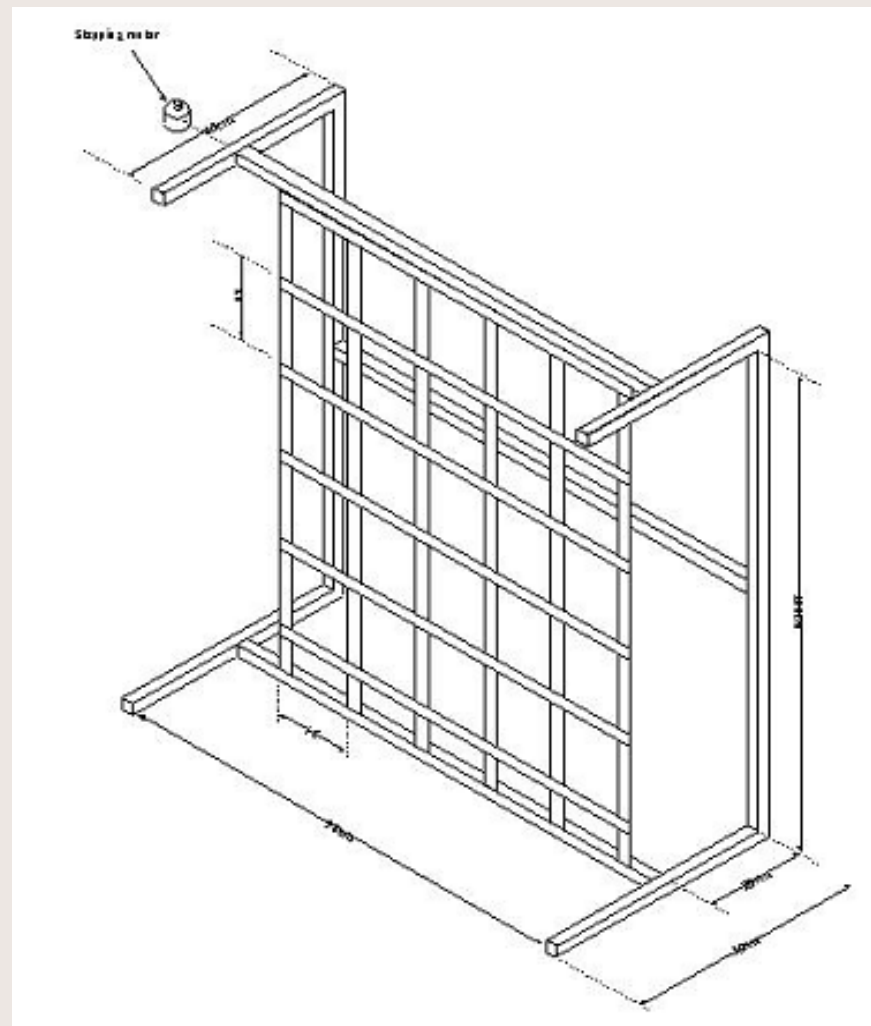
แผ่นวงจร Crane Interface



การออกแบบโครงสร้างของ Crane



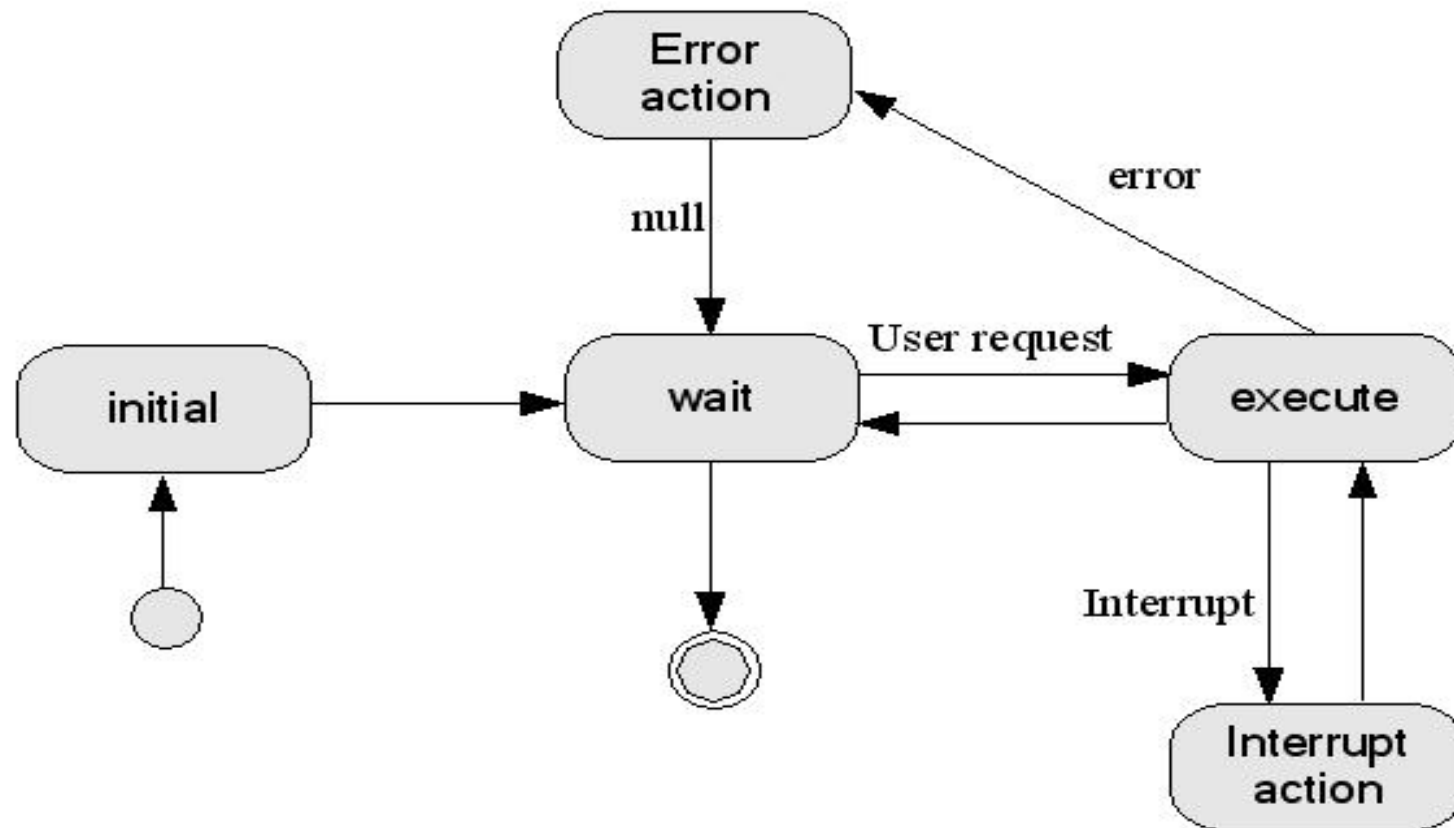
การออกแบบโครงสร้างของ Rack



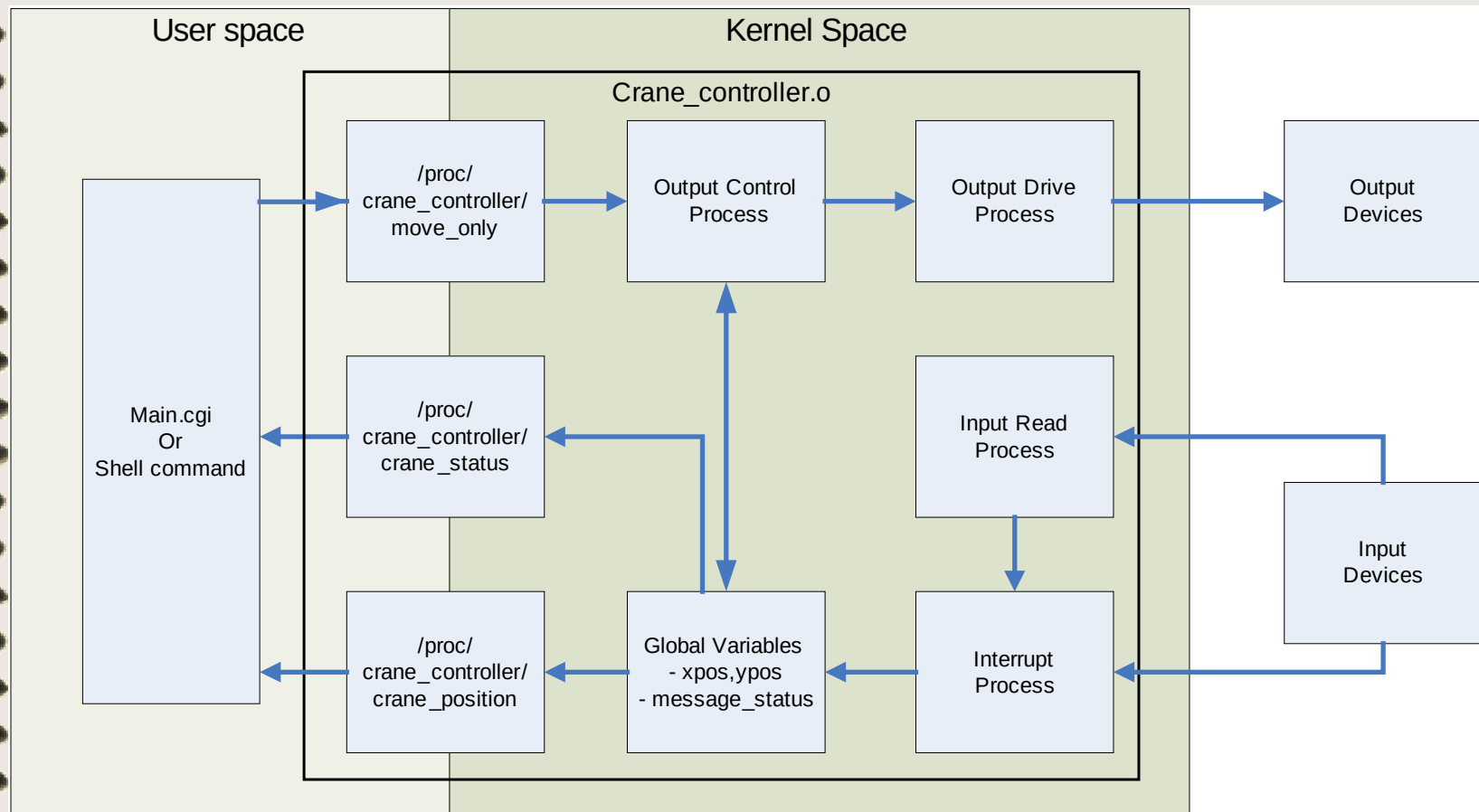
Crane และ Rack ที่ประกอบสมบูรณ์



State Diagram



โครงสร้างของ Module



Module source code

- ส่วนกำหนดตัวแปรเริ่มต้น File/Directory structor
- การทำงานขณะ Initialize / Cleanup Module
- การสร้าง Process file
- การทำงานของ Process file

โครงสร้าง Directory และ Files

```
static struct proc_dir_entry *main_dir,  
  
                                /*main process directory */  
  
*move_only_file,              /* move motor only process*/  
*get_position_file,           /* read crane position process*/  
  
*get_status_file; /* read all crane status process*/
```

การทำงานขณะ initialize Module

```
static int __init init_crane_controller(void)
{
    int rv = 0; int i;

    /* Create the crane_controller /proc entry */

    main_dir = proc_mkdir("crane_controller", NULL);

    if(main_dir == NULL) { rv = -ENOMEM;
        goto out; }
```

การสร้าง Input Process file

```
get_position_file = create_proc_entry("crane_position", 0444,  
    main_dir);  
  
if(get_position_file == NULL) {  
    rv = -ENOMEM; goto no_get_position;  
}  
  
get_position_file->read_proc = &proc_read_position;  
  
get_position_file->write_proc = NULL;  
  
get_position_file->owner = THIS_MODULE;
```

การสร้าง Output Process file

```
move_only_file = create_proc_entry("move_only", 0666, main_dir);  
if(move_only_file == NULL) {  
    rv = -ENOMEM;  
    goto no_move_only;  
}  
move_only_file->read_proc = NULL;  
move_only_file->write_proc = &proc_move_only;  
move_only_file->owner = THIS_MODULE;
```


การสร้าง Interrupt Process file

```
crane_interrupt_file = create_proc_entry("crane_interrupt", 0444,  
    main_dir);  
  
if(crane_interrupt_file == NULL) {  
    return -ENOMEM;  
}  
  
crane_interrupt_file->read_proc = NULL;  
  
crane_interrupt_file->write_proc = NULL;  
  
crane_interrupt_file->owner = THIS_MODULE;
```

การสร้าง Interrupt Process file (ต่อ)

```
/* request interrupt from linux */  
rv = request_irq(INTERRUPT, crane_interrupt_interrupt, 0,  
                "crane_interrupt",NULL);  
if ( rv ) {  
    printk("Can't get interrupt %d\n", INTERRUPT);  
    goto no_interrupt;  
}
```

การทำงานของ Input Process file

```
static int proc_read_position(char *page, char **start, off_t off, int
    count, int *eof, void *data)
{
    page[0] = (char) (xpos + 48);
    page[1] = (char) (ypos + 48);
    page[2] = '\n';
    return 3;
}
```

การทำงานของ Output Process file

```
static int proc_move_only(struct file *file, const char *buffer,  
                           unsigned long count, void *data)
```

```
...
```

```
if(count > 3)
```

```
...
```

```
tempx = chartoint(buffer[0]); /*set x target to tempx */
```

```
tempy = chartoint(buffer[1]); /*set y target to tempy */
```

การทำงานของ Output Process file (ต่อ)

```
while(!((tempx == xpos) && (tempy == ypos))&&(timeout <
MAXTIMEOUTDELAY))    /* wait for x and y position */
```

```
    mydelay(MAXMOTORDELAY);
```

```
...
```

```
    mydelaytimer = jiffies;
```

```
    while((jiffies - mydelaytimer) < mytimer) ;
```

การทำงานของ Interrupt Process file

```
void crane_interrupt_interrupt(int irq, void *dev_id, struct
pt_regs *regs) {
...
ipA = inb(SPPDATAPORT);

    if(ipA & 0x80)                                /*bottom limit switch */
    {
        ydirect = 'n';
        ypos = 0;
    }
```

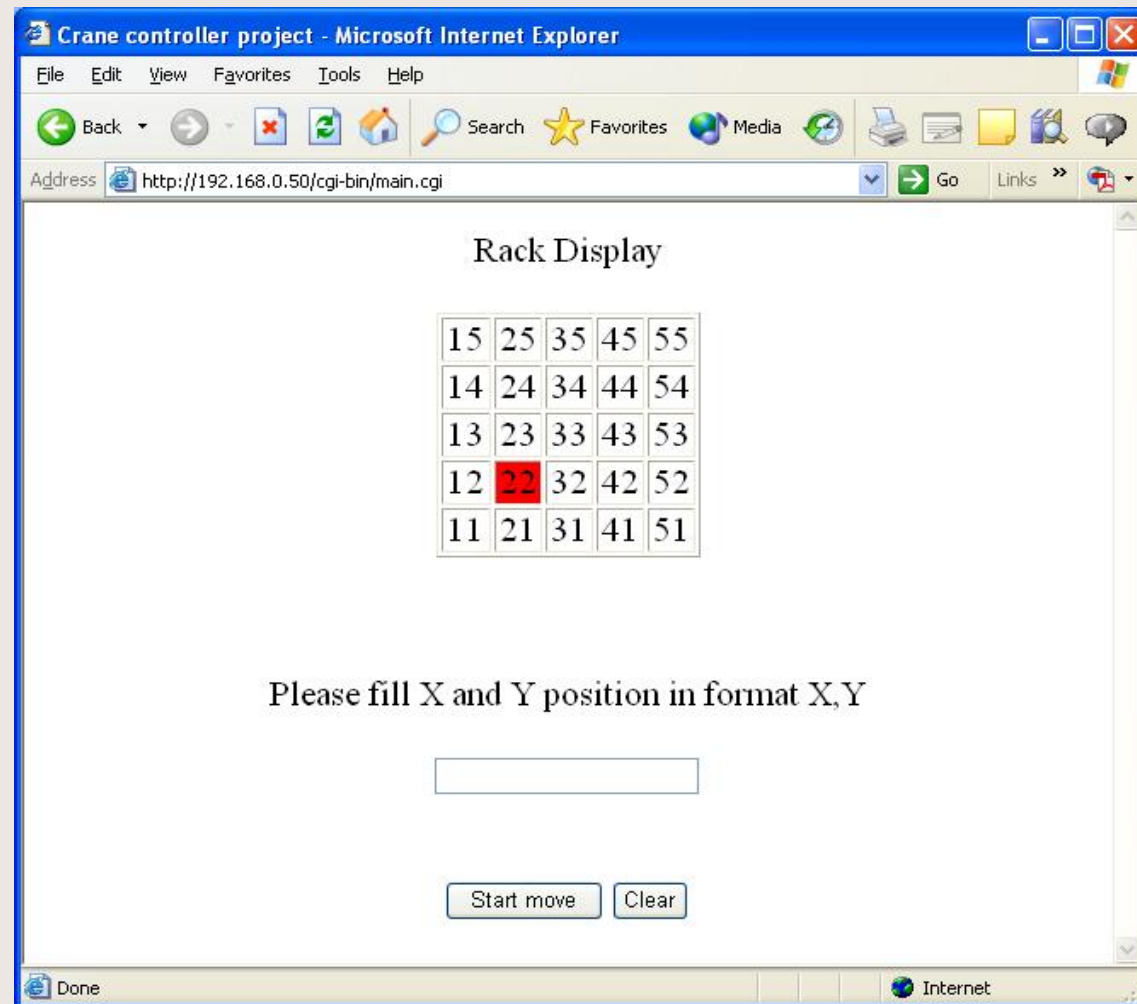
การทำงานขณะ Cleanup Module

```
static void __exit cleanup_crane_controller(void) {  
    free_irq(INTERRUPT,NULL);  
    remove_proc_entry("crane_interrupt",main_dir);  
    remove_proc_entry("crane_position",main_dir);  
    ...  
    remove_proc_entry("crane_controller", NULL);  
    printk(KERN_INFO "%s %s removed\n", MODULE_NAME,  
            MODULE_VERSION);  
}
```

ผลการทดสอบ

ระบบสามารถทำงานตามความต้องการได้ในระดับหนึ่ง กล่าวคือ เมื่อเปิด Server ระบบสามารถจัดการเครนให้อยู่ในตำแหน่งเริ่มต้น บริเวณด้านซ้ายล่างสุดของ Rack ได้ทุกครั้ง และสามารถสั่งเครนให้เคลื่อนที่ไปยังเป้าหมายที่กำหนดโดยผ่าน Web page ได้ รวมไปถึงสามารถแก้ปัญหาที่อาจเกิดขึ้นในระหว่างทำงานได้ เช่น เมื่อเครนไปไม่ถึงเป้าหมายในเวลาที่กำหนด เครนจะเคลื่อนตัวเองมายังตำแหน่ง 0,0 ทุกครั้ง เป็นต้น

ผลการทดสอบ (ต่อ)



ปัญหาที่ได้รับการแก้ไขแล้ว

- ปัญหาเกี่ยวกับโปรแกรม
 - วิธีการห้วงเวลาต้องใช้ตัวแปร *jiffies*
 - สัญญาณ *input* เกิด *oscillate* แก้ไขโดยใช้การจับ *Pulse* แรกและห้วงเวลา
- ปัญหาเกี่ยวกับ Crane interface
 - ปัญหาการสร้าง *interrupt pulse* แก้ปัญหาโดยใช้ตัวเก็บประจุช่วยในการสร้าง *pulse*

ปัญหาที่ได้รับการแก้ไขแล้ว (ต่อ)

- ปัญหาเกี่ยวกับ Crane และ Rack
 - สายพานหลุดจาก *pulley* ของ *stepping motor* แก้ไขโดยทำแผ่นครอบสายพาน และ นำยางพาราทาบริเวณส่วนของ *pulley*
 - *Stepping motor* แนวนอนรับน้ำหนักมากเกินไป แก้ไขโดยเพิ่มแรงดันให้กับ *stepping motor* เป็น 24 V

ปัญหาที่ยังไม่ได้รับการแก้ไข

- ปัญหาเกี่ยวกับโปรแกรม
 - การแสดง *Status* ยังไม่สามารถแสดงผลได้อย่างถูกต้อง
- ปัญหาเกี่ยวกับ Crane interface
 - วงจรสร้าง *interrupt pulse* จะผิดพลาดกรณีที่เกิด *interrupt* ในเวลาที่ใกล้เคียงกัน
- ปัญหาเกี่ยวกับ Crane และ Rack
 - โครงไม่ได้มาตรฐานเกิดการเอียงในแนวนอนและแนวตั้ง
 - ความเร็วของ *Motor* ต่ำเกินไป

สรุป

จากการทดสอบจะเห็นว่า Linux มีความคล่องตัวเป็นอย่างมากในการประยุกต์ใช้งานด้านต่างๆ นอกจากนี้ยังทำให้ทราบถึงขั้นตอนและวิธีการสร้าง Linux Device Driver ตั้งแต่เริ่มต้น จนสามารถใช้งานได้ ถึงแม้ว่าการทดสอบจะเป็นการเขียนโปรแกรมเพื่อควบคุมอุปกรณ์อย่างง่าย แต่ทำให้ผู้จัดทำได้แนวคิดและวิธีการในการนำ Embedded Linux ไปพัฒนาระบบที่ซับซ้อนมากขึ้นได้ในอนาคต

ข้อเสนอแนะ

แนวทางในการทำงานของผู้สนใจทางด้าน Embedded Linux ควรมีเป้าหมายในการนำไปใช้งานอย่างชัดเจนว่าจะนำไปใช้งานด้านใด และควรมีอุปกรณ์ที่ต้องการทดสอบอยู่แล้วเพื่อที่สามารถใช้เวลาในการพัฒนาโปรแกรมได้มากขึ้น และ อุปกรณ์ที่ใช้ในการทดสอบควรแสดงให้เห็นถึงประสิทธิภาพของ Embedded Linux System มากกว่านี้